

Технически университет – София

TECH WAREHOUSE

Курсова работа

Изготвил: Петър Жан Иванов

Фак. №: 121223054

Група: 44б

Дисциплина: Програмни езици

София, 18 декември 2025 г.

Резюме

Проектът TECH WAREHOUSE е конзолно приложение за управление на складови наличности за технологични продукти. Поддържат се три типа артикули: Laptop, Phone и DesktopComputer, реализирани чрез наследяване от абстрактен клас Product. Потребителят работи чрез меню с операции: добавяне, списък, търсене по име/сериен номер, редакция, изтриване, филтър по категория, Stock In/Stock Out и запис на данни. Данните се съхраняват устойчиво във файл warehouse.json чрез библиотеката nlohmann/json.

Съдържание

Резюме	1
1 Въведение	4
1.1 Проблем	4
1.2 Цел	4
1.3 Обхват	4
2 Използвани технологии и инструменти	5
2.1 Език и стандарт	5
2.2 Външна библиотека	5
2.3 Инструменти	5
3 Структура на проекта	6
4 Схема на връзките между класовете (UML)	8
4.1 Класова диаграма	8
4.2 Описание на диаграмата	9
5 Работа с приложението (Потребителско ръководство)	11
5.1 Стартиране	11
5.2 Меню и функции	11
5.3 Добавяне на продукт (Option 1)	12
5.4 Търсене (Option 3 и 4)	12
5.5 Редакция (Option 5)	12
5.6 Stock операции (Option 8 и 9)	12
5.7 Запис на данните (Option 11)	12
6 Данни и формат на warehouse.json	13
6.1 Структура	13
6.2 Примерни данни	13

7	Реализация на проекта (обяснение и код)	15
7.1	Стартова точка (TechWarehouse.cpp)	15
7.1.1	Обяснение	15
7.1.2	Код	15
7.2	Базов модел: Product (абстрактен клас)	16
7.2.1	Обяснение	16
7.2.2	Код (Product.h)	16
7.2.3	Обяснение (имплементация)	18
7.2.4	Код (Product.cpp)	18
7.3	Наследник: Laptop	19
7.3.1	Обяснение	19
7.3.2	Код (Laptop.cpp)	20
7.4	Наследник: DesktopComputer	21
7.4.1	Обяснение	21
7.4.2	Код (DesktopComputer.cpp)	22
7.5	Inventory: бизнес логика и persistence	23
7.5.1	Обяснение	23
7.5.2	Код (Inventory.h)	23
7.5.3	Обяснение (ключови моменти в Inventory.cpp)	25
7.5.4	Код (Inventory.cpp)	25
7.6	ConsoleMenu: потребителски интерфейс	32
7.6.1	Обяснение	32
7.6.2	Код (ConsoleMenu.h)	33
7.6.3	Код (ConsoleMenu.cpp)	34
8	Тестови сценарии и гранични случаи	49
9	Заклучение	50

Глава 1

Въведение

1.1 Проблем

При складова работа (дори малък склад) най-честите проблеми са: трудна проследимост на наличности, грешки при промяна на количества, липса на бързо търсене по критерии и риск от дублирани артикули. TECH WAREHOUSE адресира тези проблеми чрез централизирана структура от данни, валидирани операции и надеждно съхранение в JSON файл.

1.2 Цел

Целта е да се изгради стабилно C++ приложение, което:

- пази списък с продукти, разделени по категории;
- гарантира уникалност чрез сериен номер;
- позволява CRUD операции и търсене;
- реализира наличности (Stock In/Out) с проверки;
- зарежда/записва информацията от/в warehouse.json.

1.3 Обхват

Включва: конзолен интерфейс, модели с наследяване, инвентарна логика, JSON persistence.

Не включва: база данни, GUI, потребители/ролева система, мрежови функции.

Глава 2

Използвани технологии и инструменти

2.1 Език и стандарт

C++ със стандартни библиотеки: `vector`, `string`, `memory`, `algorithm`, `fstream`, `limits`.

2.2 Външна библиотека

- `nlohmann/json` – сериализация и десериализация на данни в JSON.

2.3 Инструменти

Visual Studio и PlantUML (за UML диаграма).

Глава 3

Структура на проекта

Проектът е организиран в отделни класове и файлове (.h/.cpp), които разделят отговорностите:

- Product + наследници – данни, тип, JSON конверсия;
- Category – категориен модел (id, име, описание);
- Inventory – бизнес логика и persistence;
- ConsoleMenu – UI и вход от потребителя;
- TechWarehouse.cpp – стартова точка (main).

```
1 Header Files :
2   Category.h
3   ConsoleMenu.h
4   DesktopComputer.h
5   Inventory.h
6   Laptop.h
7   Phone.h
8   Product.h
9
10 Source Files :
11   Category.cpp
12   ConsoleMenu.cpp
13   DesktopComputer.cpp
14   Inventory.cpp
15   Laptop.cpp
16   Phone.cpp
17   Product.cpp
18   TechWarehouse.cpp
19
```

```
20      :  
21      warehouse.json  
22      packages.config
```

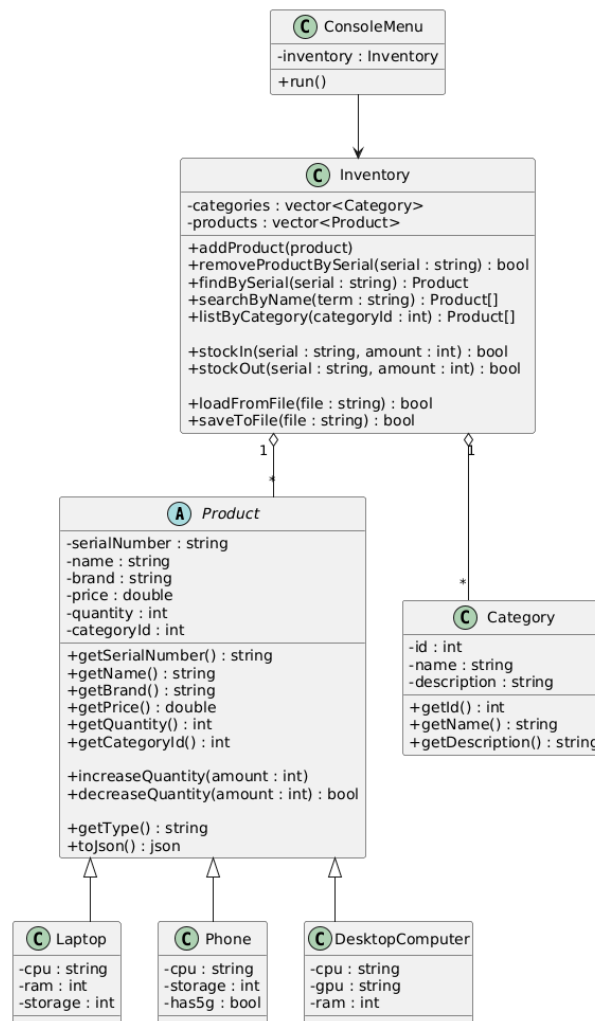
Listing 3.1: Файлова структура (основни файлове)

Глава 4

Схема на връзките между класовете (UML)

4.1 Класова диаграма

Фигура 4.2 представя класовата диаграма на проекта TECH WAREHOUSE. Диаграмата визуализира архитектурата на системата, основните класове и връзките между тях, включително наследяване и асоциации.



Фигура 4.1: Enter Caption

Фигура 4.2: UML класова диаграма на проекта TECH WAREHOUSE

4.2 Описание на диаграмата

- Абстрактният клас Product дефинира общите характеристики на всички продукти.
- Класовете Laptop, Phone и DesktopComputer наследяват Product и добавят специфични за типа полета.
- Класът Inventory поддържа колекции от продукти и категории и реализира бизнес логиката.
- Класът ConsoleMenu осигурява комуникацията между потребителя и инвентара.

- Връзките тип has-a (агрегация) показват, че Inventory управлява множество продукти и категории.

Глава 5

Работа с приложението (Потребителско ръководство)

5.1 Стартиране

При стартиране приложението:

1. създава Inventory;
2. опитва да зареди warehouse.json;
3. стартира ConsoleMenu;
4. при изход записва текущите данни обратно в warehouse.json.

5.2 Меню и функции

Менюто предлага следните операции:

1. Add product
2. List all products
3. Search product by NAME
4. Search product by SERIAL
5. Edit product
6. Remove product
7. List products by CATEGORY
8. Stock IN

9. Stock OUT
10. List categories
11. Save data
12. Exit

5.3 Добавяне на продукт (Option 1)

Добавянето включва валидации:

- serialNumber: задължителен и уникален;
- name: задължително;
- price: > 0 ;
- quantity: ≥ 0 ;
- категория: избор от списък (Laptop/Phone/Desktop).

След избор на категория типът се задава автоматично и се въвеждат специфични параметри (CPU/RAM/Storage/GPU/5G).

5.4 Търсене (Option 3 и 4)

By NAME: частично съвпадение (substring).

By SERIAL: точно съвпадение по сериен номер.

5.5 Редакция (Option 5)

Редакцията показва текущите данни и позволява промяна на базови и специфични полета според типа продукт. Завършва със Save & exit.

5.6 Stock операции (Option 8 и 9)

Stock In: amount > 0 и продуктът да съществува.

Stock Out: amount > 0 , продуктът да съществува и amount $\leq$ наличното количество.

5.7 Запис на данните (Option 11)

Записва се JSON файл warehouse.json. При успех се извежда [OK] Data saved..

Глава 6

Данни и формат на warehouse.json

6.1 Структура

Файлът съдържа JSON обект с масив "products":

- Всеки продукт има "type" (напр. Laptop/Phone/DesktopComputer).
- Общи полета: serialNumber, name, brand, price, quantity, categoryId.
- Специфични полета зависят от типа.

6.2 Примерни данни

```
1 {  
2   "products": [  
3     {  
4       "type": "Laptop",  
5       "serialNumber": "LAP-0001",  
6       "name": "Lenovo ThinkPad T14",  
7       "brand": "Lenovo",  
8       "price": 2499.99,  
9       "quantity": 5,  
10      "categoryId": 1,  
11      "cpu": "Intel Core i7-1260P",  
12      "ramGB": 16,  
13      "storageGB": 512  
14    },  
15    {  
16      "type": "DesktopComputer",  
17      "serialNumber": "DESK-9000",  
18      "name": "Gaming PC",
```

```
19     "brand": "Custom",
20     "price": 3999.00,
21     "quantity": 2,
22     "categoryId": 3,
23     "cpu": "AMD Ryzen 7 7800X3D",
24     "gpu": "NVIDIA RTX 4070",
25     "ramGB": 32
26   }
27 ]
28 }
```

Listing 6.1: Примерен warehouse.json

Глава 7

Реализация на проекта (обяснение и код)

7.1 Стартова точка (TechWarehouse.cpp)

7.1.1 Обяснение

Файлът TechWarehouse.cpp управлява жизнения цикъл на приложението:

- създава инвентара;
- зарежда данните от warehouse.json (ако файлът липсва/е невалиден, се започва с празен инвентар);
- стартира менюто;
- при изход от менюто записва данните обратно във файла.

Така се гарантира, че потребителските промени се запазват без допълнителни стъпки.

7.1.2 Код

```
1 #include <iostream>
2 #include "Inventory.h"
3 #include "ConsoleMenu.h"
4
5 int main() {
6     Inventory inventory;
7
8     if (!inventory.loadFromFile("warehouse.json")) {
9         std::cout << "Failed to load data file. Starting with empty
            inventory.\n";
10    }
11 }
```



```
12     ConsoleMenu menu(inventory);
13     menu.run();
14
15     if (!inventory.saveToFile("warehouse.json")) {
16         std::cout << "Failed to save data file.\n";
17     }
18
19     return 0;
20 }
```

Listing 7.1: TechWarehouse.cpp (main)

7.2 Базов модел: Product (абстрактен клас)

7.2.1 Обяснение

Product е абстрактен базов клас, който описва общите характеристики на всеки продукт:

- идентификация чрез serialNumber;
- описателни данни: name, brand;
- стойност и наличност: price, quantity;
- категория: categoryId.

Също така дефинира общи операции за промяна на количество (increaseQuantity, decreaseQuantity) и изисква всеки наследник да дефинира:

- getType() – връща типа на продукта като текст;
- toJson() – сериализация към JSON.

7.2.2 Код (Product.h)

```
1 #pragma once
2
3 #include <string>
4 #include <nlohmann/json.hpp>
5
6 class Product {
7 protected:
8     std::string serialNumber;
```

```
9      std::string name;
10     std::string brand;
11     double price;
12     int quantity;
13     int categoryId;
14
15 public:
16     Product(
17         const std::string& serialNumber ,
18         const std::string& name,
19         const std::string& brand ,
20         double price ,
21         int quantity ,
22         int categoryId
23     );
24
25     virtual ~Product() = default;
26
27     // Getters
28     const std::string& getSerialNumber() const;
29     const std::string& getName() const;
30     const std::string& getBrand() const;
31     double getPrice() const;
32     int getQuantity() const;
33     int getCategoryId() const;
34
35     // Setters
36     void setName(const std::string& name);
37     void setBrand(const std::string& brand);
38     void setPrice(double price);
39     void setQuantity(int quantity);
40
41     void increaseQuantity(int amount);
42     bool decreaseQuantity(int amount);
43
44     virtual std::string getType() const = 0;
45     virtual nlohmann::json toJson() const = 0;
46 };
```

Listing 7.2: Product.h

7.2.3 Обяснение (имплементация)

Имплементацията пази логика за безопасни промени:

- `setPrice` и `setQuantity` не приемат отрицателни стойности;
- `increaseQuantity` работи само при `amount > 0`;
- `decreaseQuantity` отказва при `amount <= 0` или `amount > quantity`.

Това предотвратява некоректни наличности и „минус“ количества.

7.2.4 Код (Product.cpp)

```
1 #include "Product.h"
2
3 Product::Product(
4     const std::string& serialNumber,
5     const std::string& name,
6     const std::string& brand,
7     double price,
8     int quantity,
9     int categoryId
10 )
11     : serialNumber(serialNumber),
12     name(name),
13     brand(brand),
14     price(price),
15     quantity(quantity),
16     categoryId(categoryId) {
17 }
18
19 const std::string& Product::getSerialNumber() const { return
    serialNumber; }
20 const std::string& Product::getName() const { return name; }
21 const std::string& Product::getBrand() const { return brand; }
22 double Product::getPrice() const { return price; }
23 int Product::getQuantity() const { return quantity; }
24 int Product::getCategoryId() const { return categoryId; }
25
26 void Product::setName(const std::string& newName) {
27     name = newName;
28 }
29
```

```
30 void Product::setBrand(const std::string& newBrand) {
31     brand = newBrand;
32 }
33
34 void Product::setPrice(double newPrice) {
35     if (newPrice >= 0)
36         price = newPrice;
37 }
38
39 void Product::setQuantity(int newQuantity) {
40     if (newQuantity >= 0)
41         quantity = newQuantity;
42 }
43
44 void Product::increaseQuantity(int amount) {
45     if (amount > 0)
46         quantity += amount;
47 }
48
49 bool Product::decreaseQuantity(int amount) {
50     if (amount <= 0 || amount > quantity)
51         return false;
52     quantity -= amount;
53     return true;
54 }
```

Listing 7.3: Product.cpp

7.3 Наследник: Laptop

7.3.1 Обяснение

Класът Laptop разширява Product със специфични за лаптоп параметри:

- cpu – процесор;
- ramGB – RAM в GB;
- storageGB – storage в GB.

getType() връща “Laptop”, което се използва при запис/зареждане и при визуализация. Методът toJson() връща JSON с общите полета и специфичните за типа.

7.3.2 Код (Laptop.cpp)

```
1 #include "Laptop.h"
2
3 Laptop::Laptop(
4     const std::string& serialNumber ,
5     const std::string& name,
6     const std::string& brand ,
7     double price ,
8     int quantity ,
9     int categoryId ,
10    const std::string& cpu ,
11    int ramGB,
12    int storageGB
13 )
14 : Product(serialNumber , name, brand , price , quantity ,
15          categoryId) ,
16   cpu(cpu) ,
17   ramGB(ramGB) ,
18   storageGB(storageGB) {
19
20 const std::string& Laptop::getCpu() const {
21     return cpu;
22 }
23
24 int Laptop::getRamGB() const {
25     return ramGB;
26 }
27
28 int Laptop::getStorageGB() const {
29     return storageGB;
30 }
31
32 std::string Laptop::getType() const {
33     return "Laptop";
34 }
35
36 nlohmann::json Laptop::toJson() const {
37     return {
38         {"type", getType()},
39         {"serialNumber", serialNumber},
```

```
40     {"name", name},
41     {"brand", brand},
42     {"price", price},
43     {"quantity", quantity},
44     {"categoryId", categoryId},
45     {"cpu", cpu},
46     {"ramGB", ramGB},
47     {"storageGB", storageGB}
48 };
49 }
50
51 Laptop Laptop::fromJson(const nlohmann::json& j) {
52     return Laptop(
53         j.at("serialNumber").get<std::string>(),
54         j.at("name").get<std::string>(),
55         j.at("brand").get<std::string>(),
56         j.at("price").get<double>(),
57         j.at("quantity").get<int>(),
58         j.at("categoryId").get<int>(),
59         j.at("cpu").get<std::string>(),
60         j.at("ramGB").get<int>(),
61         j.at("storageGB").get<int>()
62     );
63 }
```

Listing 7.4: Laptop.cpp

7.4 Наследник: DesktopComputer

7.4.1 Обяснение

DesktopComputer описва настолен компютър и добавя:

- cpu – процесор;
- gpu – видео карта;
- ramGB – RAM.

Типът е “DesktopComputer” и се записва в JSON, за да може Inventory да зареди правилния клас.

7.4.2 Код (DesktopComputer.cpp)

```
1 #include "DesktopComputer.h"
2
3 DesktopComputer::DesktopComputer(
4     const std::string& serialNumber ,
5     const std::string& name,
6     const std::string& brand ,
7     double price ,
8     int quantity ,
9     int categoryId ,
10    const std::string& cpu ,
11    const std::string& gpu ,
12    int ramGB
13 )
14 : Product(serialNumber , name, brand , price , quantity ,
15         categoryId) ,
16   cpu(cpu) ,
17   gpu(gpu) ,
18   ramGB(ramGB) {
19
20 std::string DesktopComputer::getType() const {
21     return "DesktopComputer";
22 }
23
24 nlohmann::json DesktopComputer::toJson() const {
25     return {
26         {"type", getType()} ,
27         {"serialNumber", serialNumber} ,
28         {"name", name} ,
29         {"brand", brand} ,
30         {"price", price} ,
31         {"quantity", quantity} ,
32         {"categoryId", categoryId} ,
33         {"cpu", cpu} ,
34         {"gpu", gpu} ,
35         {"ramGB", ramGB}
36     };
37 }
38
39 DesktopComputer DesktopComputer::fromJson(const nlohmann::json& j) {
```

```
40     return DesktopComputer(  
41         j.at("serialNumber").get<std::string>(),  
42         j.at("name").get<std::string>(),  
43         j.at("brand").get<std::string>(),  
44         j.at("price").get<double>(),  
45         j.at("quantity").get<int>(),  
46         j.at("categoryId").get<int>(),  
47         j.at("cpu").get<std::string>(),  
48         j.at("gpu").get<std::string>(),  
49         j.at("ramGB").get<int>()  
50     );  
51 }
```

Listing 7.5: DesktopComputer.cpp

7.5 Inventory: бизнес логика и persistence

7.5.1 Обяснение

Inventory е основният “service” слой. Пази:

- категории (предефинирани в конструктора);
- продукти като `std::shared_ptr<Product>` за полиморфизъм.

Реализира операции:

- добавяне/изтриване по serial;
- търсене по serial и по име;
- филтър по категория;
- Stock In/Out с отделни проверки;
- зареждане/запис в JSON.

7.5.2 Код (Inventory.h)

```
1 #pragma once  
2  
3 #include <vector>  
4 #include <memory>  
5 #include <string>
```



```
6 #include <nlohmann/json.hpp>
7
8 #include "Category.h"
9 #include "Product.h"
10
11 class Inventory {
12 private:
13     std::vector<Category> categories;
14     std::vector<std::shared_ptr<Product>> products;
15
16 public:
17     Inventory();
18
19     // ----- Categories -----
20     const std::vector<Category>& getCategories() const;
21     const Category* getCategoryById(int id) const;
22
23     // ----- Products -----
24     void addProduct(const std::shared_ptr<Product>& product);
25
26     bool serialExists(const std::string& serial) const;
27     std::shared_ptr<Product> findBySerial(const std::string& serial)
28         const;
29
30     std::vector<std::shared_ptr<Product>> getAllProducts() const;
31     std::vector<std::shared_ptr<Product>> searchByName(const
32         std::string& term) const;
33     std::vector<std::shared_ptr<Product>> listByCategory(int
34         categoryId) const;
35
36     bool removeProductBySerial(const std::string& serial);
37
38     bool updateProductFromJson(const std::string& currentSerial,
39         const nlohmann::json& updatedJson);
40
41     // ----- Validations / Stock -----
42     bool canStockIn(const std::string& serial, int amount) const;
43     bool canStockOut(const std::string& serial, int amount) const;
44
45     bool stockIn(const std::string& serial, int amount);
46     bool stockOut(const std::string& serial, int amount);
```

```
43
44 // ----- Persistence -----
45 bool loadFromFile(const std::string& file);
46 bool saveToFile(const std::string& file);
47 };
```

Listing 7.6: Inventory.h

7.5.3 Обяснение (ключови моменти в Inventory.cpp)

Важни решения:

- Уникалност: serialExists() се използва преди add/update.
- Безопасност: canStockIn/canStockOut са отделени проверки.
- Редакция: updateProductFromJson подменя продукта с нов, създаден от fromJson.
- JSON: при load се чете products масив и по type се избира наследник.

7.5.4 Код (Inventory.cpp)

```
1 #include "Inventory.h"
2
3 #include <fstream>
4 #include <algorithm>
5 #include <nlohmann/json.hpp>
6
7 #include "Laptop.h"
8 #include "Phone.h"
9 #include "DesktopComputer.h"
10
11 using json = nlohmann::json;
12
13 static std::string getStringAny(const json& j,
14     std::initializer_list<const char*> keys)
15 {
16     for (auto k : keys)
17     {
18         if (j.contains(k) && j[k].is_string())
19             return j[k].get<std::string>();
20     }
21     return "";
```

```
21 }
22
23 // ===== CONSTRUCTOR =====
24
25 Inventory::Inventory() {
26     categories.emplace_back(1, "Laptop", "Portable computers");
27     categories.emplace_back(2, "Phone", "Mobile phones");
28     categories.emplace_back(3, "Desktop", "Desktop computers");
29 }
30
31 // ===== CATEGORIES =====
32
33 const std::vector<Category>& Inventory::getCategories() const {
34     return categories;
35 }
36
37 const Category* Inventory::getCategoryById(int id) const {
38     for (const auto& c : categories) {
39         if (c.getId() == id)
40             return &c;
41     }
42     return nullptr;
43 }
44
45 // ===== PRODUCTS =====
46
47 void Inventory::addProduct(const std::shared_ptr<Product>& product) {
48     if (!product)
49         return;
50
51     if (serialExists(product->getSerialNumber()))
52         return;
53
54     products.push_back(product);
55 }
56
57 bool Inventory::serialExists(const std::string& serial) const {
58     if (serial.empty())
59         return false;
60
61     for (const auto& p : products) {
```

```
62         if (p->getSerialNumber() == serial)
63             return true;
64     }
65     return false;
66 }
67
68 std::shared_ptr<Product> Inventory::findBySerial(const std::string&
69     serial) const {
69     if (serial.empty())
70         return nullptr;
71
72     for (const auto& p : products) {
73         if (p->getSerialNumber() == serial)
74             return p;
75     }
76     return nullptr;
77 }
78
79 std::vector<std::shared_ptr<Product>> Inventory::getAllProducts()
80     const {
81     return products;
82 }
83
84 std::vector<std::shared_ptr<Product>> Inventory::searchByName(const
85     std::string& term) const {
86     std::vector<std::shared_ptr<Product>> result;
87
88     if (term.empty())
89         return result;
90
91     for (const auto& p : products) {
92         if (p->getName().find(term) != std::string::npos) {
93             result.push_back(p);
94         }
95     }
96     return result;
97 }
98
99 std::vector<std::shared_ptr<Product>> Inventory::listByCategory(int
100     categoryId) const {
101     std::vector<std::shared_ptr<Product>> result;
```

```
99
100     if (!getCategoryById(categoryId))
101         return result;
102
103     for (const auto& p : products) {
104         if (p->getCategoryId() == categoryId) {
105             result.push_back(p);
106         }
107     }
108     return result;
109 }
110
111 bool Inventory::removeProductBySerial(const std::string& serial) {
112     if (serial.empty())
113         return false;
114
115     auto it = std::remove_if(
116         products.begin(),
117         products.end(),
118         [&](const std::shared_ptr<Product>& p) {
119             return p->getSerialNumber() == serial;
120         }
121     );
122
123     if (it == products.end())
124         return false;
125
126     products.erase(it, products.end());
127     return true;
128 }
129
130 bool Inventory::updateProductFromJson(const std::string&
    currentSerial, const json& updatedJson)
131 {
132     auto existing = findBySerial(currentSerial);
133     if (!existing)
134         return false;
135
136     std::string newSerial = getStringAny(updatedJson, {
        "serialNumber", "serial", "SerialNumber", "Serial" });
137     if (newSerial.empty())
```

```
138         newSerial = existing->getSerialNumber();
139
140     if (newSerial != currentSerial && serialExists(newSerial))
141         return false;
142
143     std::string type = getStringAny(updatedJson, { "type", "Type" });
144     if (type.empty())
145         type = existing->getType();
146
147     std::shared_ptr<Product> newP;
148
149     try
150     {
151         if (type == "Laptop")
152             newP =
153                 std::make_shared<Laptop>(Laptop::fromJson(updatedJson));
154         else if (type == "Phone")
155             newP =
156                 std::make_shared<Phone>(Phone::fromJson(updatedJson));
157         else if (type == "DesktopComputer")
158             newP =
159                 std::make_shared<DesktopComputer>(DesktopComputer::fromJson(updatedJson));
160         else
161             return false;
162     }
163     catch (...)
164     {
165         return false;
166     }
167
168     for (auto& p : products)
169     {
170         if (p && p->getSerialNumber() == currentSerial)
171         {
172             p = newP;
173             return true;
174         }
175     }
176
177     return false;
178 }
```

```
176
177 // ===== VALIDATIONS =====
178
179 bool Inventory::canStockIn(const std::string& serial, int amount)
180     const {
181     if (serial.empty())
182         return false;
183
184     if (amount <= 0)
185         return false;
186
187     auto product = findBySerial(serial);
188     if (!product)
189         return false;
190
191     return true;
192 }
193
194 bool Inventory::canStockOut(const std::string& serial, int amount)
195     const {
196     if (serial.empty())
197         return false;
198
199     if (amount <= 0)
200         return false;
201
202     auto product = findBySerial(serial);
203     if (!product)
204         return false;
205
206     if (amount > product->getQuantity())
207         return false;
208
209     return true;
210 }
211
212 // ===== STOCK (DEFENSIVE) =====
213
214 bool Inventory::stockIn(const std::string& serial, int amount) {
215     if (!canStockIn(serial, amount))
216         return false;
```

```
215
216     auto product = findBySerial(serial);
217     product->increaseQuantity(amount);
218     return true;
219 }
220
221 bool Inventory::stockOut(const std::string& serial, int amount) {
222     if (!canStockOut(serial, amount))
223         return false;
224
225     auto product = findBySerial(serial);
226     return product->decreaseQuantity(amount);
227 }
228
229 // ===== JSON =====
230
231 bool Inventory::loadFromFile(const std::string& file) {
232     std::ifstream in(file);
233     if (!in.is_open())
234         return false;
235
236     json j;
237     try {
238         in >> j;
239     }
240     catch (...) {
241         return false;
242     }
243
244     products.clear();
245
246     if (!j.contains("products") || !j["products"].is_array())
247         return true;
248
249     for (const auto& item : j["products"]) {
250         if (!item.contains("type"))
251             continue;
252
253         try {
254             std::string type = item["type"].get<std::string>();
255
```



```
256         if (type == "Laptop") {
257             products.push_back(std::make_shared<Laptop>(Laptop::fromJson(item)));
258         }
259         else if (type == "Phone") {
260             products.push_back(std::make_shared<Phone>(Phone::fromJson(item)));
261         }
262         else if (type == "DesktopComputer") {
263             products.push_back(std::make_shared<DesktopComputer>(DesktopComp
264         )
265     }
266     catch (...) {
267         continue;
268     }
269 }
270
271 return true;
272 }
273
274 bool Inventory::saveToFile(const std::string& file) {
275     json j;
276     j["products"] = json::array();
277
278     for (const auto& p : products) {
279         j["products"].push_back(p->toJson());
280     }
281
282     std::ofstream out(file);
283     if (!out.is_open())
284         return false;
285
286     out << j.dump(4);
287     return true;
288 }
```

Listing 7.7: Inventory.cpp

7.6 ConsoleMenu: потребителски интерфейс

7.6.1 Обяснение

ConsoleMenu е UI слой. Неговата работа е:

- да показва меню и да приема избор;
- да валидира вход (числа, граници, празни стойности);
- да извиква методи на Inventory;
- да показва резултат/грешка.

Използва помощни функции:

- `clearInput()` за стабилно четене (`cin + getline`);
- цветни съобщения под Windows чрез WinAPI (иначе безцветен режим).

7.6.2 Код (ConsoleMenu.h)

```
1 #pragma once
2
3 #include "Inventory.h"
4
5 class ConsoleMenu {
6 private:
7     Inventory& inventory;
8
9     // ----- Core -----
10    void showMenu() const;
11
12    // ----- Products -----
13    void addProduct();
14    void listProducts();
15    void searchByName();
16    void searchBySerial();
17    void editProduct();
18    void removeProduct();
19
20    // ----- Categories -----
21    void listCategories();
22    void listByCategory();
23
24    // ----- Stock -----
25    void stockIn();
26    void stockOut();
27
28    // ----- Persistence -----
```

```
29     void saveData();
30
31 public:
32     explicit ConsoleMenu(Inventory& inventory);
33     void run();
34 };
```

Listing 7.8: ConsoleMenu.h

7.6.3 Код (ConsoleMenu.cpp)

```
1 #include "ConsoleMenu.h"
2
3 #include <iostream>
4 #include <limits>
5
6 #include "Laptop.h"
7 #include "Phone.h"
8 #include "DesktopComputer.h"
9
10 #ifdef _WIN32
11 #define NOMINMAX
12 #include <windows.h>
13 #endif
14
15 // ===== INPUT HELPERS =====
16
17 static void clearInput()
18 {
19     std::cin.clear();
20     std::cin.ignore(std::numeric_limits<std::streamsize>::max(),
21                     '\n');
22 }
23
24 // ===== CONSOLE COLORS (Windows) =====
25
26 #ifdef _WIN32
27 static WORD g_defaultAttr = 0;
28 static bool g_initiated = false;
29
30 static void initConsoleColors()
```

```
30 {
31     if (g_inited) return;
32     g_inited = true;
33
34     HANDLE h = GetStdHandle(STD_OUTPUT_HANDLE);
35     CONSOLE_SCREEN_BUFFER_INFO info;
36     if (GetConsoleScreenBufferInfo(h, &info))
37         g_defaultAttr = info.wAttributes;
38     else
39         g_defaultAttr = FOREGROUND_RED | FOREGROUND_GREEN |
40             FOREGROUND_BLUE;
41 }
42 static void setColor(WORD attr)
43 {
44     initConsoleColors();
45     SetConsoleTextAttribute(GetStdHandle(STD_OUTPUT_HANDLE), attr);
46 }
47
48 static void resetColor()
49 {
50     initConsoleColors();
51     SetConsoleTextAttribute(GetStdHandle(STD_OUTPUT_HANDLE),
52         g_defaultAttr);
53 }
54 #else
55 static void setColor(int) {}
56 static void resetColor() {}
57 #endif
58
59 // Color presets
60 static void printTitle(const std::string& text)
61 {
62     #ifdef _WIN32
63         setColor(FOREGROUND_GREEN | FOREGROUND_BLUE |
64             FOREGROUND_INTENSITY);
65     #endif
66     std::cout << "\n===== \n";
67     std::cout << text << "\n";
68     std::cout << "===== \n";
69     resetColor();
70 }
```

```
68 }
69
70 static void printOk(const std::string& text)
71 {
72     #ifdef _WIN32
73         setColor(FOREGROUND_GREEN | FOREGROUND_INTENSITY);
74     #endif
75     std::cout << "[OK] " << text << "\n";
76     resetColor();
77 }
78
79 static void printError(const std::string& text)
80 {
81     #ifdef _WIN32
82         setColor(FOREGROUND_RED | FOREGROUND_INTENSITY);
83     #endif
84     std::cout << "[ERROR] " << text << "\n";
85     resetColor();
86 }
87
88 static void printInfo(const std::string& text)
89 {
90     #ifdef _WIN32
91         setColor(FOREGROUND_RED | FOREGROUND_GREEN |
92                 FOREGROUND_INTENSITY);
93     #endif
94     std::cout << text << "\n";
95     resetColor();
96 }
97 // ===== PRODUCT PRINT =====
98
99 static void printProductDetailed(const Inventory& inv, const
100     std::shared_ptr<Product>& p)
101 {
102     if (!p) return;
103
104     const Category* cat = inv.getCategoryById(p->getCategoryId());
105     std::string catName = cat ? cat->getName() : "Unknown";
106     #ifdef _WIN32
```

```

107     setColor(FOREGROUND_RED | FOREGROUND_GREEN | FOREGROUND_BLUE |
108             FOREGROUND_INTENSITY);
109 #endif
109     std::cout << " -----\n";
110 #ifdef _WIN32
111     resetColor();
112 #endif
113
114     std::cout << " Serial    : " << p->getSerialNumber() << "\n";
115     std::cout << " Type      : " << p->getType() << "\n";
116     std::cout << " Name       : " << p->getName() << "\n";
117     std::cout << " Brand      : " << p->getBrand() << "\n";
118     std::cout << " Price      : " << p->getPrice() << "\n";
119     std::cout << " Quantity   : " << p->getQuantity() << "\n";
120     std::cout << " Category  : " << catName << " (ID=" <<
121         p->getCategoryId() << ")\n";
122
123 #ifdef _WIN32
124     setColor(FOREGROUND_RED | FOREGROUND_GREEN | FOREGROUND_BLUE |
125             FOREGROUND_INTENSITY);
126 #endif
127     std::cout << " -----\n";
128     resetColor();
129 }
130
131 // ===== CATEGORY CHOOSER =====
132
133 static int chooseCategoryId(const Inventory& inv)
134 {
135     const auto& cats = inv.getCategories();
136     if (cats.empty())
137     {
138         printError("No categories available.");
139         return -1;
140     }
141
142     printTitle("CHOOSE CATEGORY");
143     for (size_t i = 0; i < cats.size(); i++)
144         std::cout << (i + 1) << ". " << cats[i].getName() << "\n";
145
146     std::cout << "Choice (1-" << cats.size() << "): ";

```

```
145     int choice;
146     std::cin >> choice;
147
148     if (std::cin.fail())
149     {
150         clearInput();
151         printError("Invalid input.");
152         return -1;
153     }
154
155     clearInput();
156
157     if (choice < 1 || choice >(int)cats.size())
158     {
159         printError("Invalid category choice.");
160         return -1;
161     }
162
163     return cats[choice - 1].getId();
164 }
165
166 // ===== CTOR =====
167
168 ConsoleMenu::ConsoleMenu(Inventory& inventory)
169     : inventory(inventory)
170 {
171 #ifdef _WIN32
172     initConsoleColors();
173 #endif
174 }
175
176 // ===== MENU =====
177
178 void ConsoleMenu::showMenu() const
179 {
180     printTitle("TECH WAREHOUSE");
181
182     std::cout << "1.  Add product\n";
183     std::cout << "2.  List all products\n";
184     std::cout << "3.  Search product by NAME\n";
185     std::cout << "4.  Search product by SERIAL\n";
```

```
186     std::cout << "5.  Edit product\n";
187     std::cout << "6.  Remove product\n";
188     std::cout << "7.  List products by CATEGORY\n";
189     std::cout << "8.  Stock IN\n";
190     std::cout << "9.  Stock OUT\n";
191     std::cout << "10. List categories\n";
192     std::cout << "11. Save data\n";
193     std::cout << "0.  Exit\n";
194
195 #ifdef _WIN32
196     setColor(FOREGROUND_RED | FOREGROUND_GREEN |
197             FOREGROUND_INTENSITY);
198 #endif
199     std::cout << "Choice: ";
200     resetColor();
201 }
202
203 void ConsoleMenu::run()
204 {
205     int choice = -1;
206
207     while (choice != 0)
208     {
209         showMenu();
210         std::cin >> choice;
211
212         if (std::cin.fail())
213         {
214             clearInput();
215             printError("Invalid input.");
216             continue;
217         }
218
219         clearInput();
220
221         switch (choice)
222         {
223             case 1: addProduct(); break;
224             case 2: listProducts(); break;
225             case 3: searchByName(); break;
226             case 4: searchBySerial(); break;
```



```
226         case 5: editProduct(); break;
227         case 6: removeProduct(); break;
228         case 7: listByCategory(); break;
229         case 8: stockIn(); break;
230         case 9: stockOut(); break;
231         case 10: listCategories(); break;
232         case 11: saveData(); break;
233         case 0: printOk("Exiting..."); break;
234         default: printError("Unknown option."); break;
235     }
236 }
237 }
238
239 // ===== CATEGORIES =====
240
241 void ConsoleMenu::listCategories()
242 {
243     const auto& cats = inventory.getCategories();
244     if (cats.empty())
245     {
246         printError("No categories.");
247         return;
248     }
249
250     printTitle("CATEGORIES");
251     for (const auto& c : cats)
252         std::cout << c.getId() << " | " << c.getName() << "\n";
253 }
254
255 // ===== PRODUCTS =====
256
257 void ConsoleMenu::listProducts()
258 {
259     const auto& products = inventory.getAllProducts();
260     if (products.empty())
261     {
262         printInfo("No products.");
263         return;
264     }
265
266     printTitle("ALL PRODUCTS");
```

```
267     for (const auto& p : products)
268         printProductDetailed(inventory , p);
269 }
270
271 void ConsoleMenu::addProduct()
272 {
273     std::string serial , name, brand;
274     double price;
275     int quantity;
276
277     printTitle("ADD PRODUCT");
278
279     std::cout << "Serial number: ";
280     std::getline(std::cin , serial);
281
282     if (serial.empty())
283     {
284         printError("Serial number cannot be empty.");
285         return;
286     }
287
288     if (inventory.findBySerial(serial))
289     {
290         printError("Product with this serial already exists.");
291         return;
292     }
293
294     std::cout << "Name: ";
295     std::getline(std::cin , name);
296     if (name.empty())
297     {
298         printError("Name cannot be empty.");
299         return;
300     }
301
302     std::cout << "Brand: ";
303     std::getline(std::cin , brand);
304
305     std::cout << "Price: ";
306     std::cin >> price;
307     if (std::cin.fail() || price <= 0)
```

```
308     {
309         clearInput();
310         printError("Invalid price.");
311         return;
312     }
313
314     std::cout << "Quantity: ";
315     std::cin >> quantity;
316     if (std::cin.fail() || quantity < 0)
317     {
318         clearInput();
319         printError("Invalid quantity.");
320         return;
321     }
322     clearInput();
323
324     int categoryId = chooseCategoryId(inventory);
325     if (categoryId == -1) return;
326
327     const Category* cat = inventory.getCategoryById(categoryId);
328     std::string catName = cat ? cat->getName() : "";
329
330     std::shared_ptr<Product> product;
331
332     if (categoryId == 1 || catName == "Laptop")
333     {
334         printInfo("Selected: Laptop (type is set automatically)");
335
336         std::string cpu;
337         int ram, storage;
338
339         std::cout << "CPU: ";
340         std::getline(std::cin, cpu);
341
342         std::cout << "RAM (GB): ";
343         std::cin >> ram;
344         std::cout << "Storage (GB): ";
345         std::cin >> storage;
346
347         if (std::cin.fail() || ram <= 0 || storage <= 0)
348         {
```

```
349         clearInput();
350         printError("Invalid laptop specs.");
351         return;
352     }
353     clearInput();
354
355     product = std::make_shared<Laptop>(serial, name, brand,
        price, quantity, categoryId, cpu, ram, storage);
356 }
357 else if (categoryId == 2 || catName == "Phone")
358 {
359     printInfo("Selected: Phone (type is set automatically)");
360
361     std::string cpu;
362     int storage;
363     int has5g;
364
365     std::cout << "CPU: ";
366     std::getline(std::cin, cpu);
367
368     std::cout << "Storage (GB): ";
369     std::cin >> storage;
370     std::cout << "5G support (1=yes,0=no): ";
371     std::cin >> has5g;
372
373     if (std::cin.fail() || storage <= 0 || (has5g != 0 && has5g
        != 1))
374     {
375         clearInput();
376         printError("Invalid phone specs.");
377         return;
378     }
379     clearInput();
380
381     product = std::make_shared<Phone>(serial, name, brand,
        price, quantity, categoryId, cpu, storage, has5g == 1);
382 }
383 else if (categoryId == 3 || catName == "Desktop")
384 {
385     printInfo("Selected: Desktop (type is set automatically)");
386 }
```

```
387         std::string cpu, gpu;
388         int ram;
389
390         std::cout << "CPU: ";
391         std::getline(std::cin, cpu);
392         std::cout << "GPU: ";
393         std::getline(std::cin, gpu);
394
395         std::cout << "RAM (GB): ";
396         std::cin >> ram;
397
398         if (std::cin.fail() || ram <= 0)
399         {
400             clearInput();
401             printError("Invalid desktop specs.");
402             return;
403         }
404         clearInput();
405
406         product = std::make_shared<DesktopComputer>(serial, name,
407             brand, price, quantity, categoryId, cpu, gpu, ram);
408     }
409     else
410     {
411         printError("Unknown category / cannot map category to
412             product type.");
413         return;
414     }
415
416     inventory.addProduct(product);
417     printOk("Product added successfully.");
418 }
419
420 void ConsoleMenu::searchByName()
421 {
422     std::string term;
423     printTitle("SEARCH BY NAME");
424
425     std::cout << "Name: ";
426     std::getline(std::cin, term);
```

```
426     auto results = inventory.searchByName(term);
427     if (results.empty())
428     {
429         printInfo("No matches.");
430         return;
431     }
432
433     for (const auto& p : results)
434         printProductDetailed(inventory, p);
435 }
436
437 void ConsoleMenu::searchBySerial()
438 {
439     std::string serial;
440     printTitle("SEARCH BY SERIAL");
441
442     std::cout << "Serial: ";
443     std::getline(std::cin, serial);
444
445     auto p = inventory.findBySerial(serial);
446     if (!p)
447     {
448         printError("Product not found.");
449         return;
450     }
451
452     printProductDetailed(inventory, p);
453 }
454
455 void ConsoleMenu::removeProduct()
456 {
457     printTitle("REMOVE PRODUCT");
458
459     std::string serial;
460     std::cout << "Serial: ";
461     std::getline(std::cin, serial);
462
463     if (!inventory.removeProductBySerial(serial))
464         printError("Not found.");
465     else
466         printOk("Removed.");
```

```
467 }
468
469 void ConsoleMenu::listByCategory()
470 {
471     int categoryId = chooseCategoryId(inventory);
472     if (categoryId == -1) return;
473
474     auto products = inventory.listByCategory(categoryId);
475     if (products.empty())
476     {
477         printInfo("No products in this category.");
478         return;
479     }
480
481     printTitle("PRODUCTS BY CATEGORY");
482     for (const auto& p : products)
483         printProductDetailed(inventory, p);
484 }
485
486 void ConsoleMenu::stockIn()
487 {
488     printTitle("STOCK IN");
489
490     std::string serial;
491     int amount;
492
493     std::cout << "Serial: ";
494     std::getline(std::cin, serial);
495
496     auto p = inventory.findBySerial(serial);
497     if (!p)
498     {
499         printError("Invalid serial number (product not found).");
500         return;
501     }
502
503     std::cout << "Amount: ";
504     std::cin >> amount;
505     if (std::cin.fail())
506     {
507         clearInput();
```

```
508         printError("Invalid amount.");
509         return;
510     }
511     clearInput();
512
513     if (amount <= 0)
514     {
515         printError("Amount must be > 0.");
516         return;
517     }
518
519     inventory.stockIn(serial, amount);
520     printOk("Stock updated.");
521     printInfo("New quantity: " + std::to_string(p->getQuantity()));
522 }
523
524 void ConsoleMenu::stockOut()
525 {
526     printTitle("STOCK OUT");
527
528     std::string serial;
529     int amount;
530
531     std::cout << "Serial: ";
532     std::getline(std::cin, serial);
533
534     auto p = inventory.findBySerial(serial);
535     if (!p)
536     {
537         printError("Invalid serial number (product not found).");
538         return;
539     }
540
541     std::cout << "Amount: ";
542     std::cin >> amount;
543     if (std::cin.fail())
544     {
545         clearInput();
546         printError("Invalid amount.");
547         return;
548     }
```



```
549     clearInput();
550
551     if (amount <= 0)
552     {
553         printError("Amount must be > 0.");
554         return;
555     }
556
557     if (amount > p->getQuantity())
558     {
559         printError("Not enough quantity.");
560         printInfo("Available: " + std::to_string(p->getQuantity()));
561         return;
562     }
563
564     inventory.stockOut(serial, amount);
565     printOk("Stock updated.");
566     printInfo("New quantity: " + std::to_string(p->getQuantity()));
567 }
568
569 void ConsoleMenu::saveData()
570 {
571     if (inventory.saveToFile("warehouse.json"))
572         printOk("Data saved.");
573     else
574         printError("Save failed.");
575 }
```

Listing 7.9: ConsoleMenu.cpp

Глава 8

Тестови сценарии и гранични случаи

Таблица 8.1: Тестови сценарии

#	Стъпки	Очакван резултат
1	Стартиране без валиден warehouse.json	Изписва предупреждение и започва с празен инвентар.
2	Add product с празен serial	Отказ с грешка, продукт не се добавя.
3	Add product със serial, който вече съществува	Отказ (уникалност на serial).
4	Search by SERIAL за несъществуващ serial	Product not found.
5	Stock OUT с amount > наличност	Отказ + показва наличното количество.
6	Remove product с несъществуващ serial	Not found.
7	Save data	Създава/обновява warehouse.json.

Глава 9

Заклучение

Проектът TECH WAREHOUSE реализира пълния набор от основни функции за управление на складови наличности за технологични продукти. Използвани са ООР принципи (наследяване и полиморфизъм чрез Product и наследници), защитно програмиране при вход, както и устойчиво съхранение на данни в JSON.